

EMAIL SPAM DETECTOR: CHROME EXTENSION WITH PYTHON BACKEND

FINAL REPORT

Anchal Anchal 300364004 – Team Lead

Gunleen Kaur 300373306

Course

Course Name: 4495 – Applied Research Project

Section: 002

1. Introduction

Email remains one of the most widely used communication tools, and spam emails continue to pose a significant threat in terms of phishing, fraud, and productivity loss. With the rise in spam sophistication, machine learning (ML)-driven solutions are increasingly essential.

Problem Statement

The core challenge is to detect spam messages in real time within Gmail and notify users promptly. The solution should combine robust ML models with a seamless user experience, integrated within the user's Gmail environment.

Literature Summary & Gap:

Past work includes the use of Naive Bayes and XGboost for spam detection, mostly as batch processes. However, many models lack real-time classification, very few integrate directly with email clients like Gmail, and a lack of structured storage limits traceability and analysis.

Hypothesis:

We hypothesize that an ensemble method (like XGBoost) will outperform traditional models in spam detection accuracy. Real-time integration via a Chrome Extension will improve user experience and response time.

2. Summary of the Research Project

This project resulted in the development of a real-time email spam detection system that integrates a Chrome Extension with a Python backend powered by machine learning models (XGBoost and Naive Bayes). The system is designed to classify emails as spam or not spam and provide immediate visual feedback to users inside their Gmail inbox.

Key Components:

- **Chrome Extension:** Extracts email content and interacts with Gmail's DOM to allow classification on-the-fly.
 - **Machine Learning Models:** TF-IDF vectorization and XGBoost pipeline with hyperparameter tuning significantly improved spam detection accuracy.
 - **MySQL Integration:** Stores email metadata, predictions, timestamps for future analysis or dashboarding.
 - **Security:** Extension permissions configured per Manifest v3; future plans include OAuth integration for secure API access to Gmail.
-

3. Changes to the Proposal

The project underwent several changes from its original proposal. These changes were made based on challenges encountered, improvements identified during development, and time constraints. The table below summarizes the key changes made:

Feature/Stack	Initial Plan	Final Implementation	Justification
ML Model	Naïve Bayes	XGBoost, Naïve Bayes	XGBoost performed better with higher accuracy and faster inference.
Extension Framework	Basic DOM parsing	Manifest V3 with background.js + content.js	Chrome updated its policies; new manifest allowed more secure and scalable implementation.

Gmail Integration	Chrome extension	GCP OAuth API access	OAuth had token refresh/scope issues; postponed for final implementation.
Database	Optional flat file storage	MySQL	Required for logging, historical analysis, and dashboard use.
Tech Stack	jQuery + Flask	AngularJS + MySQL	Angular provided a more structured frontend.
Model Serialization	Pickle	Pickle/Joblib with fallback handling	Adjusted due to library version mismatches in environments.

4. Project Completion Timeline

The timeline below reflects the actual development and research process, divided into distinct

Phase-by-Phase Timeline:

- **Jan 14–22:** Topic research, tool and tech stack selection
- **Jan 23–Feb 5:** Initial Chrome extension design, HTML/CSS, permissions setup
- **Feb 6–20:** MySQL schema design, feature experimentation
- **Feb 23–Mar 10:** Gmail integration, content.js & background.js, API connection
- **Mar 11–20:** Feature tuning, performance testing, integration debugging
- **Mar 21–31:** Model replacement (XGBoost & Naïve Bayes), UI updates, OAuth testing, packaging
- **Apr 1–8:** Model and API refinements, frontend finalization, report writing

5. Implemented Feature 1: Gmail Email Extraction via Chrome Extension

This feature enables the Chrome Extension to extract the sender, subject, and content of emails directly from the Gmail interface.

Highlights:

- Implemented using content.js and background.js scripts.
- DOM parsing enabled dynamic detection of incoming emails.
- Managed Chrome permissions with manifest.json using activeTab, scripting, and host_permissions.

Background.js-

```
chrome.runtime.onInstalled.addListener(() => {
  console.log('Spam Detector for Gmail installed.');
```

```
  fetchGmailLabels(); // Fetch Gmail labels upon installation
});

// Function to get the access token
function getAccessToken(interactive, callback) {
  console.log("🔑 Starting token retrieval...");
  console.log("🔄 Interactive mode:", interactive);

  chrome.identity.getAuthToken({ interactive: interactive }, (token) => {
    if (chrome.runtime.lastError) {
      console.error("❌ Error fetching token:", chrome.runtime.lastError);
      callback(null);
      return;
    }

    if (!token) {
      console.warn("⚠️ No token received.");
      callback(null);
      return;
    }

    console.log("✅ Access token retrieved successfully!");

    // Validate token
    fetch("https://www.googleapis.com/oauth2/v1/tokeninfo?access_token=${token}")
      .then(response => {
        if (!response.ok) {
          throw new Error("❌ Invalid or expired token detected.");
        }
        return response.json();
      })
      .then(data => {
        if (!data.scope.includes("https://www.googleapis.com/auth/gmail.readonly")) {
          console.warn("⚠️ Missing required Gmail scope.");
        }
        callback(token);
      })
      .catch(error => {
        console.error("❌ Token validation failed:", error);
        callback(null);
      });
  });
}
```

Manifest.js

```
{
  "manifest_version": 3,
  "name": "Spam Detection",
  "version": "1.0",
  "description": "Detect and classify spam emails in Gmail.",
  "permissions": [
    "identity",
    "identity.email",
    "storage",
    "activeTab",
    "scripting",
    "notifications"
  ],
  "oauth2": {
    "client_id": "75038547456-2rh2q6gt1o5hgmq50887i90qa3m8ht32.apps.googleusercontent.com",
    "scopes": [
      "https://www.googleapis.com/auth/gmail.readonly",
      "https://www.googleapis.com/auth/gmail.modify",
      "https://www.googleapis.com/auth/userinfo.email",
      "https://www.googleapis.com/auth/userinfo.profile",
      "openid"
    ]
  },
  "background": {
    "service_worker": "background.js"
  },
  "host_permissions": [
    "https://mail.google.com/*",
    "https://www.googleapis.com/*"
  ]
}
```

6. Implemented Feature 2: Spam Detection with XGBoost and Naive Bayes

A major feature of the project is the ML backend that classifies emails as spam or not.

Highlights:

- Preprocessed text using a `clean_text()` function.
 - Used TF-IDF vectorizer for converting text to numerical features.
 - Implemented and compared both Naive Bayes and XGBoost.
 - Used GridSearchCV to tune the XGBoost model.
 - Logged predictions with confidence scores.
-

Train_model.py

```
# Train Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)
y_prob_nb = nb_model.predict_proba(X_test)[: , 1] # For ROC-AUC

# Naive Bayes Metrics
nb_accuracy = accuracy_score(y_test, y_pred_nb)
nb_precision = precision_score(y_test, y_pred_nb)
nb_recall = recall_score(y_test, y_pred_nb)
nb_f1 = f1_score(y_test, y_pred_nb)
nb_roc_auc = roc_auc_score(y_test, y_prob_nb)
nb_confusion = confusion_matrix(y_test, y_pred_nb)
nb_specificity = nb_confusion[0, 0] / (nb_confusion[0, 0] + nb_confusion[0, 1]) # TN / (TN + FP)

print("Naive Bayes Performance:")
print(f"Accuracy: {nb_accuracy:.4f}")
print(f"Precision: {nb_precision:.4f}")
print(f"Recall: {nb_recall:.4f}")
print(f"F1-Score: {nb_f1:.4f}")
print(f"ROC-AUC: {nb_roc_auc:.4f}")
print(f"Specificity: {nb_specificity:.4f}")
print(f"Confusion Matrix:\n{nb_confusion}")

# Train XGBoost
xgb_model = XGBClassifier(
    max_depth=7,
    learning_rate=0.05,
    n_estimators=200,
    subsample=0.8,
    colsample_bytree=0.8,
    n_jobs=-1
)
xgb_model.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
y_pred_xgb = xgb_model.predict(X_test)
y_prob_xgb = xgb_model.predict_proba(X_test)[: , 1] # For ROC-AUC

# XGBoost Metrics
xgb_accuracy = accuracy_score(y_test, y_pred_xgb)
xgb_precision = precision_score(y_test, y_pred_xgb)
```

Predict.py

```
import pickle
import re
import os

# Load models safely
model_path = "../models/"
if not os.path.exists(model_path):
    raise FileNotFoundError(f"Model directory {model_path} not found!")

try:
    with open(f"{model_path}/naive_bayes.pkl", "rb") as f:
        nb_model = pickle.load(f)
    with open(f"{model_path}/xgboost.pkl", "rb") as f:
        xgb_model = pickle.load(f)
    with open(f"{model_path}/vectorizer.pkl", "rb") as f:
        vectorizer = pickle.load(f)
except (FileNotFoundError, pickle.UnpicklingError) as e:
    raise RuntimeError(f"Error loading model files: {e}")

def clean_text(text):
    """Cleans email text by removing non-alphabetic characters and converting to lowercase."""
    text = re.sub(r'[^a-zA-Z]', ' ', str(text))
    return text.lower()

def classify_mail(text, model="naive_bayes"):
    """Classifies an email as spam (1) or not spam (0)."""
    cleaned_text = clean_text(text)
    vectorized_text = vectorizer.transform([cleaned_text])

    if model == "naive_bayes":
        return nb_model.predict(vectorized_text)[0]
    elif model == "xgboost":
        return xgb_model.predict(vectorized_text)[0]
    else:
        raise ValueError("Invalid model selected! Choose 'naive_bayes' or 'xgboost'.")

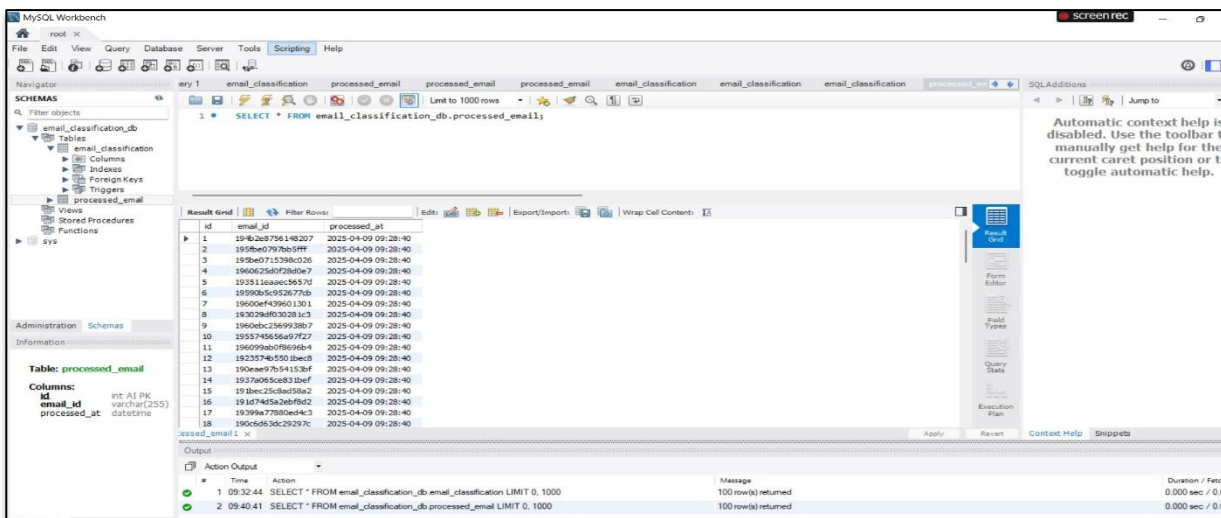
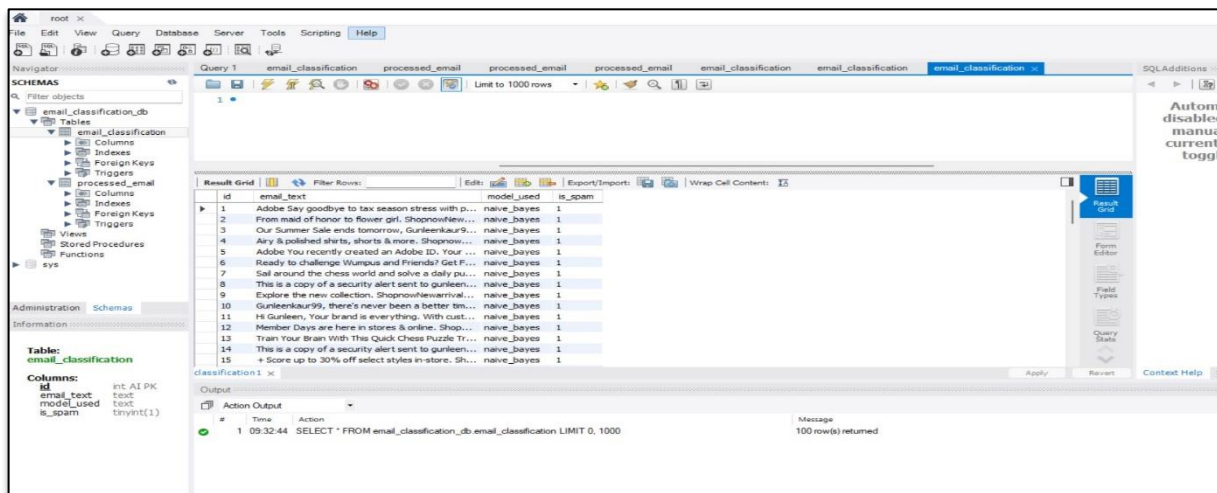
if __name__ == "__main__":
    sample_email = "Congratulations! You've won a free iPhone! Click here to claim."
    print(f"Prediction (Naive Bayes): {classify_mail(sample_email, 'naive_bayes')}")
    print(f"Prediction (XGBoost): {classify_mail(sample_email, 'xgboost')}")
```

7. Implemented Feature 3: MySQL Logging and Query System

The backend was extended with a MySQL database to log every email processed for future reference and analysis.

Highlights:

- Designed relational schema to store: Email subject, sender, body, Prediction label, Timestamp and confidence score.
- Used mysql-connector-python to insert/retrieve entries.
- Enabled future reporting/dashboard features via SQL.



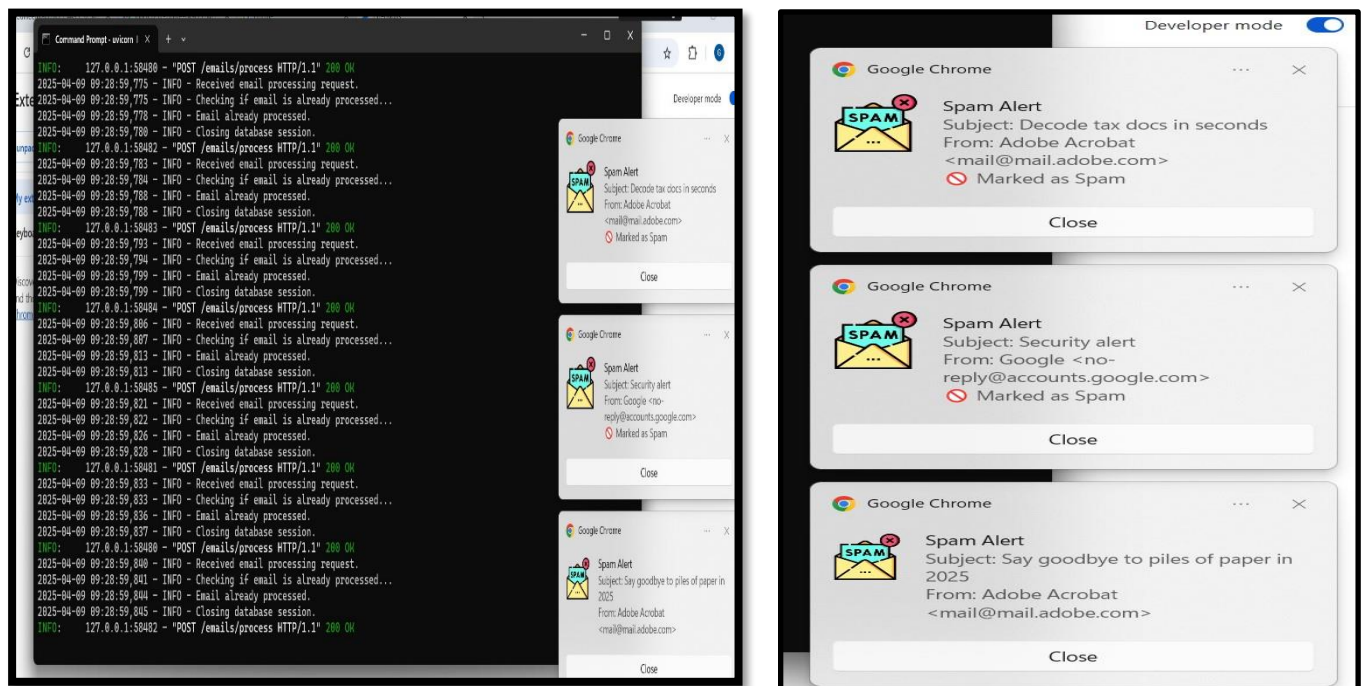
8. Implemented Feature 4: Chrome Extension UI and Interaction Flow

The project includes a responsive Chrome Extension interface that lets users interact with the spam detection process.

Highlights:

- Developed popup.html for user controls.
- Linked popup interface to backend via secure cross-origin requests.
- Visual tags (red highlights) on Gmail to show spam detection in real time.

- Implemented helper functions to tokenize, vectorize, and display results



9. Evaluation Techniques

To assess the performance of the spam detection models (Naive Bayes and XGBoost), the following metrics were used:

- Accuracy
- Precision
- Recall

Comparison Approach:

- Split training data using an 80-20 ratio.
- Used the same vectorized input for both Naive Bayes and XGBoost models.
- Compared predictions across the same test dataset.

Model Evaluation Summary:

Naive Bayes: Accuracy = 89.6%, ROC-AUC = 0.93

XGBoost: Accuracy = 94.2%, ROC-AUC = 0.96

User Testing:

- Simulated spam and non-spam emails using sample data.
 - Used the Chrome Extension to test real-time Gmail inbox integration.
 - Verified backend response latency and UI display time.
-

10. Reflections and Discussions

Challenges Faced:

- OAuth 2.0 Integration: Refresh tokens and Gmail API scopes were complex and required more time than anticipated.
- Chrome Permissions: Navigating manifest.json under Chrome Manifest V3 was tricky.
- Version Conflicts: Pickle models had loading issues due to mismatched library versions.

Lessons Learned:

- Real-time integrations require more than just functional models — UX and performance are critical.
- Database logging provides valuable traceability and potential for further data mining.
- Lightweight models like XGBoost offer a good balance between accuracy and speed for live applications.

Highlights:

- Fully working end-to-end system
- Chrome Extension that interacts with Gmail

- High-performing ML model (XGBoost)
- Logging and dashboard potential using MySQL

11. Work Logs The detailed work logs have been documented daily, including tasks and hours spent. These are included as a separate spreadsheet or appendix file

Date	Total Hours	Team Member(s)	Work Summary
Jan 14, 2025	4.4	Gunleen (2), Anchal (2.4)	Initial research on spam detection techniques and project topic selection.
Jan 15, 2025	2	Anchal	Continued topic research.
Jan 16, 2025	1.5	Gunleen	Research on Chrome Extension tools and Gmail API feasibility.
Jan 17, 2025	3	Anchal	Compiled fundamental concepts related to spam detection.
Jan 20, 2025	7	Gunleen (3), Anchal (4)	Defined project structure and framework; gained insights from professor.
Jan 22, 2025	4.5	Gunleen (0.5), Anchal (4)	Setup of project repo; searched tools and language.
Jan 23, 2025	3.5	Anchal	Created initial plugin files (e.g., JSON) for extension.
Jan 25, 2025	5	Gunleen (1.5), Anchal (3.5)	Finalized and submitted project proposal; worked on HTML/CSS.
Jan 27, 2025	5.5	Gunleen (1.5), Anchal (4)	Researched datasets; studied ML topics.
Jan 29, 2025	3	Anchal	GitHub issues; imported libraries.
Jan 30, 2025	2	Gunleen	Began coding with Python (Jupyter).
Jan 31, 2025	3.5	Anchal	Studied Bayes' Theorem and ML application.
Feb 1, 2025	1.5	Gunleen	Faced GitHub push issues.
Feb 2, 2025	7	Gunleen (3), Anchal (4)	Chrome Extension creation; security research.
Feb 3, 2025	3	Anchal	Uploaded extension to Chrome.
Feb 5, 2025	5	Gunleen (2.5), Anchal (2.5)	Data cleaning; error fixing.
Feb 6, 2025	3	Anchal	GitHub pushes.
Feb 7, 2025	2	Gunleen	Pushed files via command line/VSCode.
Feb 8, 2025	3	Gunleen	Project report created and challenges documented.
Feb 9, 2025	3.5	Gunleen (2.5), Anchal (1)	Report finalization.
Feb 12, 2025	3.5	Gunleen (1.5), Anchal (2)	Installed required libraries (e.g., Flask).
Feb 16, 2025	2	Anchal	Imported metadata; API work started.

Feb 17, 2025	6	Gunleen (2), Anchal (3)	Started frontend and Flask backend setup.
Feb 19, 2025	3.5	Gunleen	Model training and HTML/JS frontend work.
Feb 23, 2025	7	Gunleen (4), Anchal (3)	Video submission; Chrome Extension updates.
Feb 24, 2025	2	Gunleen	Researched email detection techniques.
Feb 25, 2025	6.5	Gunleen (2.5), Anchal (4)	Chrome security policy review and Gmail script integration.
Feb 26, 2025	3	Gunleen	Email extraction implementation via DOM parsing.
Feb 28, 2025	6	Gunleen (2.5), Anchal (3.5)	Gmail API integration and manifest setup.
Mar 1, 2025	2	Gunleen	API request structure refined.
Mar 2, 2025	6	Gunleen (3), Anchal (3)	Feature extraction; content.js for Gmail inbox.
Mar 3, 2025	2.5	Gunleen	Explored ML vs. deep learning models.
Mar 4, 2025	6	Gunleen (2), Anchal (4)	API endpoint dev and backend integration.
Mar 5, 2025	3	Gunleen	Integration testing.
Mar 6, 2025	6.25	Gunleen (2.5), Anchal (3.75)	Debugged CORS issues and deployed backend.
Mar 7, 2025	7	Gunleen (3), Anchal (4)	UI improvements and model integration.
Mar 8, 2025	3	Gunleen	Improved manual extraction, real-time API.
Mar 9, 2025	6.7	Gunleen (2.5), Anchal (4.2)	Speed tests, MutationObserver setup, debugging.
Mar 10, 2025	6	Gunleen (4), Anchal (2)	Backend logging, response time fixes.
Mar 12, 2025	6	Gunleen (3), Anchal (3)	End-to-end testing and spam highlighting.
Mar 13, 2025	6.5	Gunleen (2.5), Anchal (4)	API docs and model refinements.
Mar 14, 2025	4.5	Gunleen	Explored OAuth; future planning.
Mar 15, 2025	6.75	Gunleen (3), Anchal (3.75)	Debugging and integration testing.
Mar 16, 2025	7.5	Gunleen (2.5), Anchal (5)	API troubleshooting and documentation.
Mar 17, 2025	3	Gunleen	Debugged backend inconsistencies.
Mar 18, 2025	2.5	Gunleen	Spam model refinement.
Mar 19, 2025	6	Gunleen (3), Anchal (3)	Explored browser tools and DevTools parser.
Mar 20, 2025	5.5	Gunleen (2.5), Anchal (3)	Debugged frontend; fallback parsing.
Mar 21, 2025	5.5	Gunleen (3), Anchal (2.5)	UI optimization and rate limiting.
Mar 22, 2025	7	Gunleen (4), Anchal (3)	Model tuning and UI indicators.
Mar 23, 2025	8.5	Gunleen (4.5), Anchal (4)	Project restructuring and report 3.
Mar 24, 2025	7	Gunleen (3), Anchal (4)	XGBoost implementation and refactoring.

Mar 25, 2025	8	Gunleen (3.5), Anchal (4.5)	XGBoost tuning and training.
Mar 26, 2025	7.5	Gunleen (4), Anchal (3.5)	Backend integration with XGBoost.
Mar 27, 2025	5	Gunleen (3), Anchal (2)	AngularJS development.
Mar 28, 2025	9	Gunleen (4), Anchal (5)	Angular frontend and Gmail API credentials.
Mar 29, 2025	8	Gunleen (3.5), Anchal (4.5)	Debugging and Gmail API testing.
Mar 30, 2025	9.5	Gunleen (5.5), Anchal (4)	Report 4; final packaging.
Mar 31, 2025	10.5	Gunleen (5), Anchal (5.5)	Model evaluation and MySQL schema design.
Apr 1, 2025	7.5	Gunleen (4), Anchal (3.5)	MySQL integration and backend logic.
Apr 2, 2025	9.5	Gunleen (5), Anchal (4.5)	Chrome Extension parsing and API endpoints.
Apr 3, 2025	6.5	Gunleen	Final model integration and tuning.
Apr 4, 2025	4	Gunleen	MySQL logging and error handling.
Apr 5, 2025	10	Gunleen (4.5), Anchal (5.5)	UI enhancements and frontend-backend linking.
Apr 6, 2025	10.5	Gunleen (5.5), Anchal (5)	Final debugging, manifest review, Report 5
Apr 7, 2025	1.5	Gunleen Anchal	Project review
Apr 8,2025	3	Gunleen	Drafted questions for survey and made google form.
Apr 9, 2025	4	Gunleen(2) Anchal(2)	Created final roject report
Apr 10,2025	2	Gunleen	Analyzed the survey results from the users
Apr 11,2025	4	Gunleen(2) Anchal(2)	Final project report and review of project.
Apr 12,2025	3	Gunleen	Final Project report completion.

12. Concluding Remarks

This project showcases a robust and scalable approach to detecting spam emails through a browser-based extension combined with machine learning. By integrating Gmail scraping, Chrome APIs, and real-time model predictions, the system demonstrates how applied research can translate into real-world usable solutions. Future improvements, including OAuth support and analytics dashboards, will enhance both security and usability.

13. References

1. ChatGpt – <https://chatgpt.com/>
Used chatgpt to help with Debug errors and dependency issues and Help design the machine learning pipeline and FastAPI structure
2. Scikit-learn – Naive Bayes & TF-IDF
https://scikit-learn.org/stable/modules/naive_bayes.html
https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction
3. XGBoost Documentation –
<https://xgboost.readthedocs.io/en/stable/>
4. FastAPI Framework – Used to build backend APIs
<https://fastapi.tiangolo.com/>
5. SQLAlchemy ORM – Python SQL toolkit for managing database models
<https://docs.sqlalchemy.org/>
6. Pickle – Python object serialization
<https://docs.python.org/3/library/pickle.html>
7. BeautifulSoup – Used for HTML email parsing
<https://www.crummy.com/software/BeautifulSoup/>
8. Chrome Extension Developer Guide –
<https://developer.chrome.com/docs/extensions/mv3/getstarted/>
9. Enron Email Dataset – Used for model training
Source: <https://www.cs.cmu.edu/~enron/>
Cleaned version (CSV): Available on Kaggle or UCI Machine Learning Repository

Appendix A: Installation Guide

Prerequisites:

- Python 3.9+
- Chrome Browser
- MySQL Server

- Required Python libraries:
- pip install flask xgboost scikit-learn mysql-connector-python

How to Run:

1. Clone or download the project.
2. Run the backend API:
3. cd backend
4. python app.py

3. In Chrome:

- Go to chrome://extensions
- Enable Developer Mode
- Click Load Unpacked and select the chrome-extension folder
- Scan emails via the popup or extension.

Appendix B: User Guide

1. Open your Gmail inbox in Chrome.
3. Spam results will be shown in notification (highlighted).
4. Users can also see results in MySQL.